

INFYLE TECHNOLOGIES

Developer Technical Brief

Construction Workforce, Job & Site Management Platform

Backend + User Mobile App + Contractor ERP App + Admin Web Panel

Item	Specification
Document purpose	Developer-facing functional and technical implementation brief
Primary interfaces	Admin Web, User Android App, Contractor ERP Android App
Backend	NestJS / Node.js, REST API, Modular Monolith
Database	PostgreSQL
Cache / Queue	Redis
File storage	AWS S3
Push notifications	Firebase FCM
Infrastructure	AWS EC2, Ubuntu, Docker, Nginx, CloudWatch
Security	HTTPS/TLS, JWT/session auth, RBAC, validation, restricted DB/Redis access
Source basis	Signed NDA & SOW dated 18 August 2026

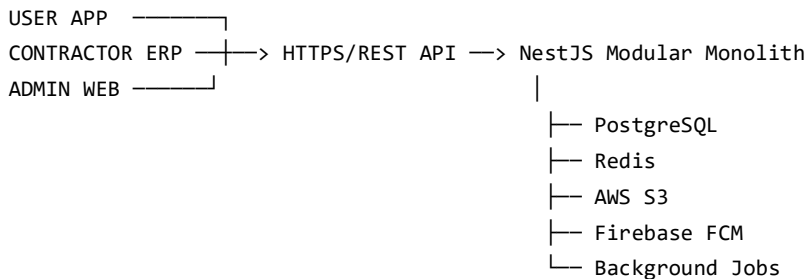
This document converts the approved SOW into an implementation-oriented brief. Where the SOW does not define an exact endpoint, table, UI control, or internal code structure, the document marks the recommendation as implementation guidance rather than a contractual feature.

1. Product Understanding — One Page

The platform is one shared backend powering three client interfaces. The User Android App handles Job Seeker, Company and construction calculation functionality. The Contractor ERP Android App handles Contractor and Site Engineer operations for projects and sites. The Admin Web Panel provides centralized administration, moderation, configuration, reporting and access control.

Interface	Primary users	Main responsibility
User Android App	Job Seeker / Company user	Jobs, applications, company job posting, profiles, construction calculators
Contractor ERP Android App	Contractor / Site Engineer	Projects, sites, attendance, labour, materials, expenses, costing, progress and daily reports
Admin Web Panel	Admin	Central control of users, companies, contractors, projects, jobs, tools, reports, notifications and permissions
Backend API	All interfaces	Authentication, authorization, business rules, persistence, files, notifications and reporting

Core business flow



The SOW explicitly defines a Modular Monolith rather than microservices for the MVP. PostgreSQL and Redis remain protected internally and are not exposed directly to the public internet.

2. Scope & Role Model

2.1 Included roles

Role	Interface	Access summary
Admin	Admin Web	Platform-wide management and RBAC
Job Seeker	User Android App	Profile, jobs, applications and notifications
Company	User Android App	Company profile, job creation, publishing and applicant management
Contractor	Contractor ERP Android App	Projects, sites, engineers, costing, reports and operational oversight
Site Engineer	Contractor ERP Android App	Only assigned project/site; daily labour, material, expense, progress and report submission

2.2 Explicit scope exclusions

- No separate Sub-Contractor application.
- No separate Sub-Contractor role.
- No separate Builder/Client Portal.
- No separate Worker Mobile Application.
- Site Engineer is a role inside the Contractor ERP, not a separate app.

Source: SOW pages 9 and 29–30. The SOW states that Job Seeker and Company functionality stays inside the User Mobile App, while project/site operations stay inside the Contractor ERP.

3. User Android App — Functional Map

3.1 Authentication

- Mobile registration
- OTP verification
- Login
- Forgot password
- Profile management

3.2 Job Seeker

Area	Developer responsibilities
Profile	Personal information, photo, skills, experience, education, work history
Preferences	Preferred location and salary expectation
Documents	Certificates, experience letters and other supported documents
Jobs	Browse, search, category/location filtering, details
Applications	Apply, save jobs, application history and status
Notifications	Job and application-related alerts

3.3 Company

Area	Developer responsibilities
Registration	Company registration and profile
Verification	Company details, contact details, verification information and documents
Job creation	Title, description, skills, location, compensation and workforce requirements
Publishing	Publish, edit and close jobs
Applicants	List applicants, view profiles/skills/experience/documents
Application control	Shortlisting and application status management

3.4 Construction Calculation Tools

Tool	Expected capability
Concrete	Construction quantity calculation based on approved formulas
Cement	Cement quantity/estimation calculation
Sand	Sand quantity/estimation calculation
Aggregate	Aggregate quantity/estimation calculation
Brick	Brick quantity calculation
Steel	Steel estimation/calculation
Flooring	Flooring quantity/estimation
Paint	Paint quantity/estimation
Plaster	Plaster quantity/estimation
Material Estimation	General material estimation

The SOW says the exact calculation logic/formulas are to be agreed and approved before implementation. Applicable configurations may be manageable through Admin where included in final requirements.

4. Contractor ERP Android App — Functional Map

4.1 Contractor

- Contractor registration/profile, company information, contact details and documents.
- Dashboard for operational/project overview.
- Create and manage projects.
- Set project duration, location and budget/contract value where applicable.
- Create one or more project sites.

- Assign Site Engineers.
- View project progress, costing, expenses and profitability.

4.2 Project Site

- Create site and store site information.
- Assign Site Engineer.
- View site progress.
- View attendance, material usage, labour cost, expenses and daily reports.

4.3 Site Engineer

The Site Engineer is the operational data-entry role. Access must be restricted to the project/site assigned to that engineer.

Daily area	Data captured
Labour	Category, worker count, attendance, labour cost, overtime, remarks
Material	Material, quantity, unit, rate, total cost, consumed quantity, stock/balance where applicable
Expenses	Category, description, amount, date, receipt/supporting document, remarks
Progress	Work completed, status, progress %, site remarks, photos/documents
Daily report	Project, site, date, labour, material, expenses, work, progress, cost, value, profit, remarks and attachments

5. Core Business Logic

5.1 Daily project cost

Daily Project Cost = Labour Cost + Material Cost + Site Expenses + Other Configured Costs

The backend should calculate the consolidated daily cost from approved records. Avoid duplicating the formula in multiple client apps; the API should remain the authoritative calculation layer.

5.2 Estimated daily profit

Estimated Daily Profit = Daily Project Value / Revenue - Daily Project Cost

The final financial formula and methodology must be approved by the Client before implementation. The SOW specifically makes this approval a dependency.

5.3 Daily report lifecycle

1. Site Engineer opens the assigned site.
2. Enters labour, attendance, material, expenses and progress.
3. Backend validates records and calculates daily cost/value/profit where applicable.
4. Engineer submits the daily report.
5. Submitted report becomes available to authorized Contractor/Admin users.
6. Notifications may be generated for report submission and configured workflow events.

6. Backend Architecture

Technology stack defined in the SOW:

Layer	Technology
API / Backend	NestJS / Node.js
Architecture	Modular Monolith
API style	REST API
Database	PostgreSQL

Cache / Queue	Redis
File storage	AWS S3
Push	Firebase Cloud Messaging
Server	AWS EC2
OS	Ubuntu Linux
Container	Docker
Reverse proxy	Nginx
Monitoring	AWS CloudWatch
Security	HTTPS / JWT / RBAC

6.1 Backend module boundaries

Module	Responsibility
auth	Registration, login, OTP, password reset, token/session handling
authorization	RBAC, role checks, permission guards
users	User profile and account lifecycle
companies	Company profile, verification and documents
contractors	Contractor profile and project access
projects	Project master data, budget, duration, location
project-sites	Sites, engineer assignment and site-level access
site-engineers	Engineer profile and assignment rules
jobs	Job creation, publishing, editing, closing and moderation
applications	Job applications, status and shortlisting
tools/calculators	Construction calculators and configurable formulas
attendance	Check-in/out, attendance history, working hours and overtime
materials	Material master, purchase, requests, issue, consumption and stock
expenses	Project/site expenses and attachments
reports	Daily/project/site/attendance/cost/profitability reports
notifications	FCM tokens, push events and announcements
documents	S3 upload metadata and secure file references
audit-logs	Security and important business activity trail

7. Recommended Backend Project Structure

The following structure is implementation guidance. The SOW defines the modules and technology, but does not prescribe exact folder names.

```

src/
  common/
    guards/      # JWT + RBAC guards
    decorators/  # Current user, roles, permissions
    filters/     # Exception handling
    interceptors/ # Logging / response handling
    pipes/       # Validation
    utils/
  config/
  modules/
    auth/
    users/
    companies/
    contractors/
    projects/
    project-sites/
    site-engineers/
    jobs/
    applications/
    calculators/
    attendance/

```

```

materials/
expenses/
reports/
notifications/
documents/
audit-logs/
database/
jobs/           # Background processing
app.module.ts
main.ts

```

7.1 Module pattern

```

module/
  controller.ts    # HTTP routes
  service.ts       # business rules
  repository.ts    # DB access (if repository abstraction is used)
  dto/
  entities/
  validators/
  tests/

```

Keep financial calculations, access rules and status transitions in backend services rather than trusting values submitted by mobile/web clients.

8. Data Model — Recommended Logical Relationships

The SOW names the business objects but does not provide a final ERD. The following is a recommended logical model for development and should be finalized during database design.

Entity	Key relationships
User	One user can have one/multiple role contexts depending on approved RBAC design
Company	Company owns/creates Jobs; company has documents and verification data
Job	Belongs to Company; receives Applications
Application	Links Job + Job Seeker; contains status and timestamps
Contractor	Owns/manages Projects
Project	Belongs to Contractor; contains one or more Project Sites
Project Site	Belongs to Project; assigned to one/more Site Engineers per final rule
Site Engineer	User identity + engineer role; access constrained by assignments
Attendance	Linked to project/site and labour/worker record
Labour Record	Linked to project/site/date; feeds labour cost
Material	Master material definition
Material Transaction	Purchase/request/issue/consumption/stock movement
Expense	Linked to project/site/date; supports attachments
Daily Report	Project + site + date; aggregates daily operational records
Document	Metadata reference to S3 object
Notification	User/device/event driven push record
Audit Log	Actor + action + entity + timestamp + metadata

8.1 Important data rules

- Use server-generated IDs and timestamps.
- Store monetary values using a fixed-precision numeric/decimal strategy rather than floating point.
- Store uploaded file metadata in PostgreSQL while binaries remain in S3.
- Do not expose raw S3 credentials to mobile apps.
- Use database transactions for multi-record operations that must remain consistent.
- Keep report snapshots/audit history if later edits could change reported financial results.

9. API Design Guidelines

The SOW specifies REST APIs but does not define exact routes. These examples are recommended conventions, not fixed contractual endpoint names.

Domain	Example endpoints
Auth	POST /auth/register, POST /auth/verify-otp, POST /auth/login, POST /auth/forgot-password
Jobs	GET /jobs, GET /jobs/:id, POST /jobs, PATCH /jobs/:id, POST /jobs/:id/close
Applications	POST /jobs/:id/apply, GET /applications/me, PATCH /applications/:id/status
Projects	GET /projects, POST /projects, GET /projects/:id, PATCH /projects/:id
Sites	POST /projects/:id/sites, GET /sites/:id, POST /sites/:id/assign-engineer
Daily ops	POST /sites/:id/labour, POST /sites/:id/materials, POST /sites/:id/expenses
Attendance	POST /sites/:id/attendance/check-in, POST /sites/:id/attendance/check-out
Reports	GET /projects/:id/reports, POST /sites/:id/daily-reports
Files	POST /documents/presign, POST /documents/complete
Notifications	POST /notifications/register-device

9.1 API response principles

- Consistent HTTP status codes.
- Consistent error object: code, message, fieldErrors where required.
- Pagination for jobs, applicants, projects, reports, notifications and large lists.
- Server-side filtering/sorting for large datasets.
- Idempotency protection for sensitive operations where duplicate submission is possible.
- Never trust client-calculated cost/profit values as authoritative.

10. Authentication & RBAC

The SOW requires HTTPS/TLS, JWT/session-based authentication, RBAC, password hashing, API/input validation and authentication rate limiting.

Layer	Rule
Authentication	Every protected API validates the user's token/session
Authorization	Role/permission guards decide whether the action is allowed
Site isolation	Site Engineer may access only assigned project/site data
Contractor isolation	Contractor access should be restricted to owned/authorized projects
Admin	Admin permissions should be centrally controlled
Input validation	Validate DTOs and business constraints server-side
Rate limiting	Protect OTP/login/auth endpoints from abuse
Files	Validate type/size/access; use secure object references

10.1 Example permission matrix

Capability	Admin	Contractor	Site Engineer	Company	Job Seeker
Manage users	Yes	No	No	No	No
Create project	Yes	Yes	No	No	No
Create site	Yes	Yes	No	No	No
Enter daily site data	Yes	Yes	Yes	No	No
Submit daily report	Yes	Yes	Yes	No	No
View project profitability	Yes	Yes	Assigned scope	No	No
Create job	Yes	No	No	Yes	No
Apply for job	Yes	No	No	No	Yes

Manage calculators	Yes	No	No	No	No
--------------------	-----	----	----	----	----

This matrix is a practical interpretation of the SOW's role model and should be finalized with the approved RBAC design.

11. File & Document Handling

AWS S3 is the file store for profile images, company/contractor/project/site documents, certificates, reports, receipts and supporting attachments.

Client App -> Backend authorization -> secure upload flow -> S3

L-> PostgreSQL stores metadata/reference

- Generate unique object keys; do not use user-provided filenames as the storage key.
- Validate MIME type and file size.
- Store owner, entity type, entity ID, object key, original filename, size and timestamps.
- Use private S3 objects and controlled access URLs where appropriate.
- Do not store large binary files directly in PostgreSQL.

12. Notifications & Background Jobs

Firebase Cloud Messaging (FCM) is specified for mobile push notifications.

Event	Potential recipient
New job	Relevant Job Seekers
Application update	Job Seeker / Company
Project assignment	Contractor / assigned engineer
Site assignment	Assigned Site Engineer
Attendance event	Relevant Contractor/Admin users
Daily report submitted	Authorized Contractor/Admin
Project update	Relevant project users
Admin announcement	Configured audience

Recommended background-job use: notification fan-out, report export generation and other non-blocking tasks. Redis is specified as cache/queue infrastructure.

13. Reporting & Analytics

Report group	Reports
Project	Summary, progress, project cost, labour cost, material cost, expense summary, profitability, daily reports
Site	Daily site, labour, material, expense, progress
Operational	Attendance, contractor, user, company, job/application
Financial	Cost and profitability
Export	PDF and Excel/CSV where included in approved implementation

For large exports, generate files asynchronously rather than blocking the API request. Store generated report files in S3 when appropriate.

14. Admin Web Panel

Module	Main functions
Dashboard	Platform, user, company, contractor, project, job and operational statistics
Users	Listing, profile, Job Seeker and Company management, status/verification

Contractors	Listing, profile, projects, status and Site Engineer management
Projects	Projects, sites, engineers, reports and operational data
Jobs	Categories, listings, moderation and applications
Tools	Calculator categories, configuration and tool management
Reports	User/company/contractor/project/site/attendance/cost/profitability
Notifications	Push notifications, announcements and alerts
RBAC	Central role and permission management

Admin Web stack specified in the SOW: React / Next.js / Tailwind CSS.

15. Mobile Front-End Architecture — Recommended

Both mobile applications are specified as Flutter applications. They should consume the same REST backend while maintaining separate app experiences.

Layer	Recommended responsibility
Presentation	Screens, widgets, navigation, loading/error/empty states
State	Auth/session state, feature state, form state
Domain	Use cases/business-facing orchestration
Data	API clients, DTO mapping, local cache where needed
Services	FCM, secure storage, file picker/camera, permissions

15.1 User App navigation

Splash -> Auth -> Role/Context -> Home
Home -> Jobs -> Job Details -> Apply -> Application Status
Home -> Company -> Jobs -> Applicants -> Applicant Details
Home -> Tools -> Calculator -> Inputs -> Result

15.2 Contractor ERP navigation

Login -> Dashboard
Dashboard -> Projects -> Project -> Sites -> Site
Site -> Attendance / Labour / Materials / Expenses / Progress
Site -> Daily Report -> Review -> Submit
Project -> Costing / Profitability / Reports
Profile -> Contractor / Engineer account settings

16. State & Status Design

Status names below are recommended implementation states. The SOW defines the business actions but not exact enum values.

Object	Recommended states
Job	Draft -> Published -> Closed
Application	Submitted -> Shortlisted -> Rejected / Accepted / Closed
Project	Draft -> Active -> Completed / Archived
Site	Active -> Inactive / Completed
Daily Report	Draft -> Submitted -> Reviewed (if review workflow is approved)
Document	Pending -> Verified / Rejected (if verification workflow is approved)

17. Infrastructure & Deployment

The SOW recommends an initial AWS EC2-based production architecture.

Component	SOW specification
-----------	-------------------

EC2	t4g.medium, 2 vCPU, 4 GB RAM, ARM64/Graviton
Storage	100–200 GB SSD
Runtime	Docker
Reverse proxy	Nginx
API	NestJS
Database	PostgreSQL
Cache/queue	Redis
Monitoring	CloudWatch
Files	AWS S3
Push	Firebase FCM

17.1 Production request path

Internet

- > HTTPS / TLS
- > Nginx
- > NestJS REST API
 - > PostgreSQL
 - > Redis
 - > AWS S3
 - > Background Jobs
 - > Firebase FCM

PostgreSQL and Redis should remain on protected/private interfaces and should not be publicly exposed.

18. Environment Configuration

Recommended configuration pattern; exact variables are implementation details.

```

NODE_ENV=production
PORT=3000
DATABASE_URL=...
REDIS_URL=...
JWT_SECRET=...
AWS_REGION=...
AWS_S3_BUCKET=...
AWS_ACCESS_KEY_ID=...
AWS_SECRET_ACCESS_KEY=...
FIREBASE_PROJECT_ID=...
FIREBASE_CLIENT_EMAIL=...
FIREBASE_PRIVATE_KEY=...

```

Secrets must be stored securely and never committed to source control or embedded in mobile binaries.

19. Security Checklist

- HTTPS/TLS enabled in production.
- JWT/session authentication implemented consistently.
- RBAC enforced at API level.
- Site Engineer scope checked against assignment on every protected resource.
- Password hashing used; plaintext passwords never stored.
- DTO/input validation enabled.
- Authentication endpoints rate-limited.
- SQL injection protected through parameterized ORM/query mechanisms.
- CORS configured intentionally.
- Secure HTTP headers configured at Nginx/application layer.

- Private PostgreSQL/Redis access only.
- S3 objects protected and access controlled.
- Audit logs for important administrative/business actions.
- Database backups configured.
- CloudWatch monitoring and operational alerts configured.

20. Testing Strategy

Test level	Minimum focus
Unit	Calculation logic, permissions, status transitions, validators
Service	Project/site access, costing, applications, materials, expenses
API	Auth, RBAC, CRUD, validation, pagination and error handling
Integration	PostgreSQL, Redis, S3 and FCM integrations
Mobile	Navigation, forms, offline/error states, file upload, push handling
Admin Web	Management screens, filters, reports, RBAC
Security	Auth abuse, unauthorized resource access, file validation
UAT	End-to-end business workflows with client-approved formulas

21. End-to-End Developer Scenarios

Scenario A — Job Seeker applies for a job

7. User registers and completes OTP verification.
8. User completes profile, skills, experience and documents.
9. User searches/filters jobs.
10. User opens job details and submits application.
11. Backend validates eligibility and creates application.
12. Company sees applicant in applicant management.
13. Company changes status/shortlists applicant.
14. Backend triggers notification to the Job Seeker.

Scenario B — Contractor creates project and site

15. Contractor logs in.
16. Creates project with duration/location/budget where applicable.
17. Creates one or more project sites.
18. Adds/assigns Site Engineer.
19. Backend records assignment and access scope.
20. Site Engineer sees only assigned site/project in ERP.

Scenario C — Site Engineer submits daily report

21. Engineer opens assigned site.
22. Records labour and attendance.
23. Records material purchase/usage/consumption.
24. Records expenses and attachments.
25. Records work progress and photos.
26. Backend calculates daily cost from approved inputs.
27. Backend calculates estimated daily profit using approved business formula.
28. Engineer submits report.
29. Contractor/Admin can view submitted report.

22. Developer Acceptance Checklist

- All three interfaces connect to the same production backend API.

- User App supports Job Seeker and Company flows.
- Construction calculators use client-approved formulas.
- Contractor ERP supports Contractor and Site Engineer roles.
- Site Engineer access is assignment-based.
- Project/site/labour/material/expense records are linked correctly.
- Daily cost is calculated server-side.
- Daily profitability is calculated server-side after formula approval.
- Daily reports can be submitted and viewed by authorized users.
- Admin can manage users, companies, contractors, projects, jobs and tools.
- Notifications are integrated through FCM.
- Files are stored in S3 with metadata in PostgreSQL.
- RBAC and validation are enforced server-side.
- PostgreSQL and Redis are not publicly exposed.
- CloudWatch monitoring is configured.
- Production deployment and source-code handover follow the SOW terms.

23. Delivery & Timeline Reference

Phase	SOW duration	Primary output
1. Requirement + UI/UX	5–7 working days	Requirements, user flows, wireframes, UI/UX, architecture
2. Backend + DB	8–10 working days	APIs, auth, RBAC, PostgreSQL, core logic
3. Mobile Apps	12–15 working days	User App + Contractor ERP
4. Admin Panel	7–10 working days	Dashboard, management, reports, configuration
5. QA + Finalization	5–8 working days	Testing, bug fixes, optimization, client review
Core development	40–50 working days	Overall development timeline
Deployment	13–15 working days	Production deployment, subject to external dependencies

Deployment timing is separate from core development and depends on AWS, domain/DNS, SSL, third-party credentials, Play Store requirements and client approvals.

24. Source-of-Truth Notes for Developers

- The signed SOW is the contractual source of truth for scope.
- Do not add a new role/app merely because it appears logically useful.
- No Sub-Contractor app or role is included.
- No separate Worker app is included.
- No separate Builder/Client portal is included.
- Site Engineer remains inside the Contractor ERP.
- Financial formulas require Client approval before final implementation.
- Third-party/server/cloud charges are separate from the development fee.
- Out-of-scope functionality must be handled as a Change Request.

25. Source References

This brief is based on the uploaded NDA & Statement of Work, especially:

Pages	Covered content
5–8	Project overview, objectives, User App, Job Seeker, Company and calculators
9–14	Contractor ERP, Site Engineer, daily operations, costing, profit,

	reports
15–17	Admin Panel, notifications and system interfaces
18–22	Technology stack, backend architecture, infrastructure, storage and security
23–25	Deliverables, timeline and development phases
29–30	Scope limitations and final system structure diagram

The final system structure diagram on page 30 visually confirms the three-interface model and the flow from projects/sites through attendance, labour, material, expenses, daily cost, daily profit, reports and notifications. The server architecture diagram on page 21 shows the API layer connecting PostgreSQL, Redis, S3, background jobs and Firebase FCM.